Mini-projet AWD | Année universitaire 2026–2027

UNIVERSITY EVENTS

Plateforme intelligente de gestion des événements universitaires

Réalisé par : Farah Shimi -Mohamed saad - wajih soussou - haithem ben massoud

SOMMAIRE

1. Présentation du projet
2. Analyse des besoins
3. Architecture technique
4. Description des microservices
5. Technologies et gestion du projet
6. Conception UML
7. Conclusion

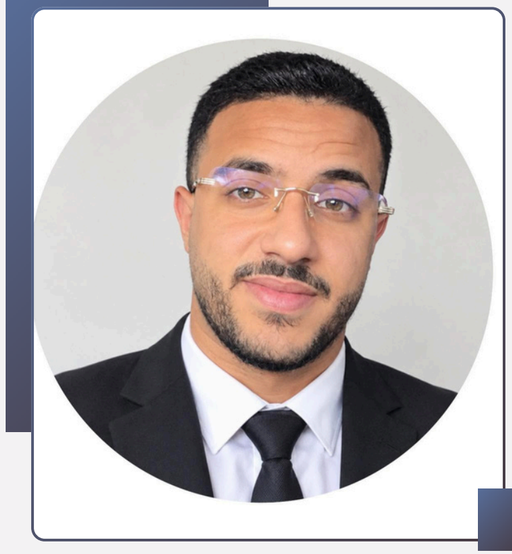
NOTRE EQUIPE



Farah Shimi



Mohamed saad



Hyathem ben Messaoud



Wajih soussou

POURQUOI UNIVERSITY EVENTS ?

La gestion des événements universitaires présente plusieurs difficultés :

- Informations dispersées : événements annoncés sur différents canaux.
- Gestion complexe : inscriptions et disponibilités difficiles à suivre.
- Manque de centralisation : absence d'une plateforme unique.
- Suivi limité : peu de statistiques pour les responsables.

NOTRE PROBLEMATIQUE :

Comment centraliser et simplifier la gestion des événements universitaires tout en garantissant une architecture évolutive, sécurisée et modulaire ?



NOTRE SOLUTION : UNIVERSITY EVENTS

Une application Web centralisée permettant de :

- **Découvrir** les événements universitaires.
- **Organiser** et publier des événements.
- **Réserver** des places en ligne.
- **Gérer** les salles et leurs capacités.
- **Évaluer** les événements.
- **Analyser** les statistiques de participation.

Notre approche : une architecture microservices pour séparer les responsabilités et faciliter l'évolution du système.

A QUI S'ADRESSE NOTRE PLATEFORME ?

Étudiant

- Consulter les événements
- S'inscrire ou annuler une inscription
- Déposer un avis

Organisateur

- Créer et modifier des événements
- Suivre les inscriptions

Administrateur

- Gérer les salles
- Administrer les événements
- Consulter les statistiques

Responsable

- Analyser les indicateurs
- Comparer les événements
- Étudier les évaluations

BESOINS FONCTIONNELS

Les besoins fonctionnels définissent les principales fonctionnalités offertes par la plateforme University Events.

Fonctionnalité	Description
Gestion des événements	CRUD des événements universitaires.
Gestion des inscriptions	Permettre aux étudiants de s'inscrire et d'annuler leurs inscriptions.
Gestion des salles	Gérer les salles et vérifier leur disponibilité.
Gestion des avis	Permettre aux étudiants inscrits d'évaluer les événements.
Statistiques	Consulter les indicateurs de participation et les évaluations.
Gestion des accès	Authentifier les utilisateurs et gérer leurs rôles.

BESOINS NON FONCTIONNELS

Les besoins non fonctionnels définissent les exigences de qualité et les contraintes techniques du système.



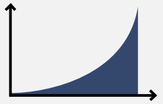
Sécurité

Authentification et gestion des rôles avec Keycloak.



Performance

Temps de réponse adaptés aux opérations courantes.



Évolutivité

Ajout de fonctionnalités grâce à l'architecture microservices.



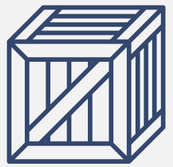
Maintenabilité

Services indépendants facilitant les mises à jour.



Fiabilité

Gestion des erreurs et vérification des opérations métier.



Portabilité

Déploiement simplifié grâce à Docker Compose.

PERIMETRE DE LA PREMIERE VERSION

Fonctionnalités incluses

- Gestion des événements
- Gestion des salles
- Inscriptions et annulations
- Avis et évaluations
- Statistiques

Fonctionnalités hors périmètre

- Paiement en ligne
- Billetterie externe
- Application mobile native

Priorité

mettre en place les fonctionnalités essentielles avant les fonctionnalités avancées.

UNE ARCHITECTURE BASEE SUR 5 MICROSERVICES

MS1 — Events

Gestion des événements : création, modification, suppression et consultation.

MS2 — Registrations

Gestion des inscriptions, annulations et participants.

MS3 — Venues

Gestion des salles, capacités et disponibilités.

MS4 — Reviews

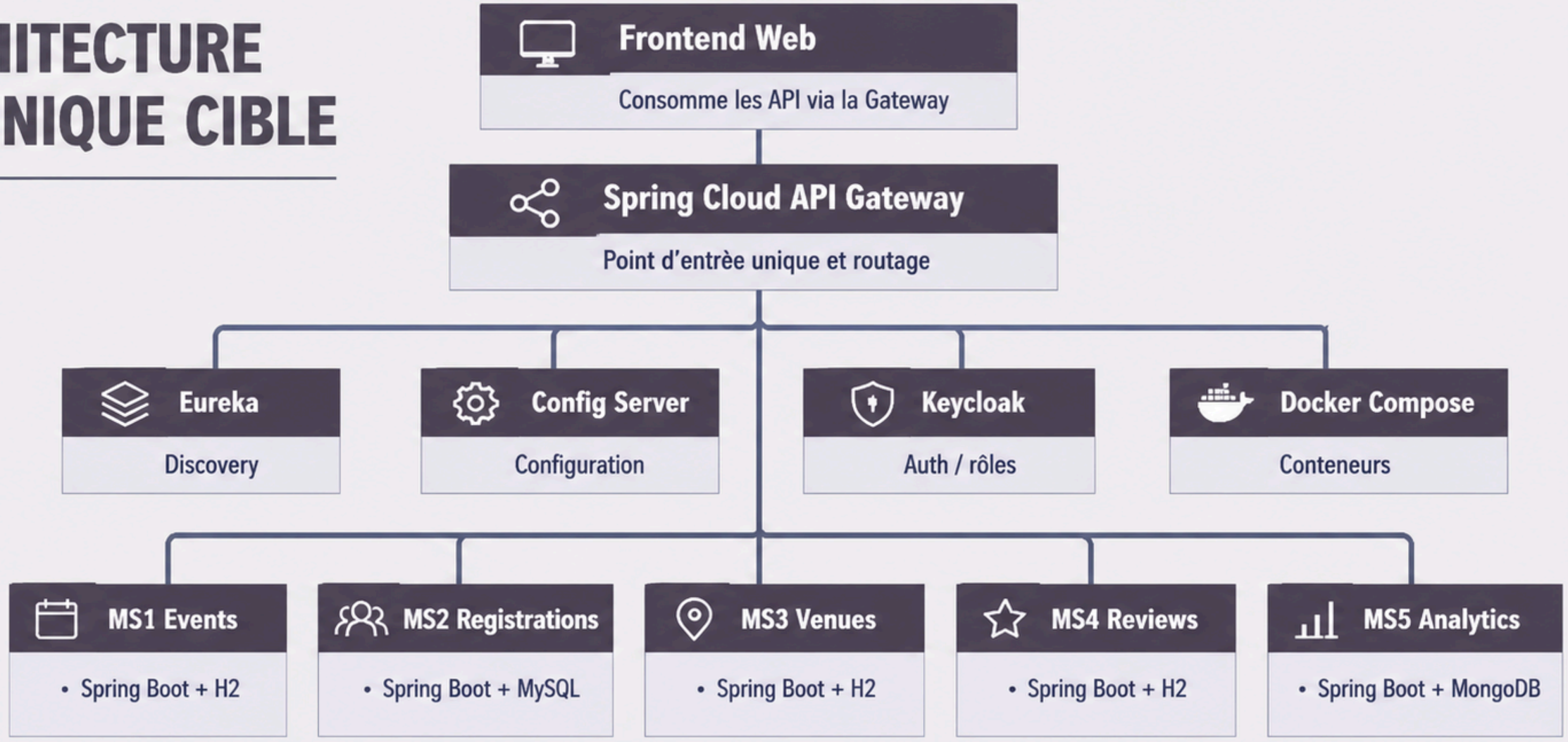
Gestion des avis, notes et évaluations.

MS5 — Analytics

Agrégation des données et production de statistiques.

Chaque microservice possède ses propres responsabilités, données et API REST.

ARCHITECTURE TECHNIQUE CIBLE



Eureka : découverte des services • **Config Server :** configuration centralisée • **Keycloak :** identité, rôles et tokens • **Docker Compose :** lancement global

UNE BASE DE DONNEES PAR MICROSERVICE

Microservice	Base proposée	Données
Events	H2	Événements
Registrations	MySQL	Inscriptions
Venues	H2	Salles
Reviews	H2	Avis
Analytics	MongoDB	Statistiques

Principe fondamental :

Chaque service est propriétaire de ses données. Les échanges entre services s'effectuent via leurs API, sans accès direct aux bases des autres services.

INTERACTIONS ENTRE MICROSERVICES

Communications synchrones (REST / OpenFeign)

MS1 Events	→	MS3 Venues	Vérifier l'existence et la disponibilité d'une salle.
MS2 Registrations	→	MS1 Events	Vérifier l'événement et les places restantes.
MS4 Reviews	→	MS2 Registrations	Vérifier que l'étudiant est inscrit.
MS5 Analytics	→	MS1 / MS2 / MS4	Récupérer les données pour les indicateurs.

Gestion des pannes | Timeout et réponse contrôlée : ne pas confirmer une opération si la vérification métier nécessaire échoue.

SECURITE ET INFRASTRUCTURE

API Gateway

- Point d'entrée centralisé
- Routage des requêtes vers les microservices

Docker Compose

- Démarrage coordonné des composants

Eureka

- Enregistrement et découverte des services

Config Server

- Centralisation des configurations

Keycloak

- Authentification des utilisateurs
- Gestion des rôles : étudiant, organisateur, administrateur et responsable
- Protection des API par tokens

❇ MS1 — EVENTS MANAGEMENT

Mission : gérer le cycle de vie des événements universitaires.

Entité principale : Event

id : Long
title : String
description : String
date : LocalDateTime
capacity : Integer
availablePlaces : Integer
venueId : Long
organizerId : String
status : String

Endpoints principaux

POST /api/events
GET /api/events
GET /api/events/{id}
PUT /api/events/{id}
DELETE /api/events/{id}
GET /api/events/{id}/availability

Interaction :
Events → Venues pour vérifier la salle avant de créer un événement.

❇ MS2 — REGISTRATION MANAGEMENT

Mission :gérer les inscriptions des étudiants aux événements.

Entité principale : Registration

id : Long
studentId : String
eventId : Long
registrationDate : LocalDateTime
status : String

Endpoints principaux

POST /api/registrations
GET /api/registrations
GET /api/registrations/{id}
GET /api/registrations/student/{id}
DELETE /api/registrations/{id}

Interaction :
Registrations → Events pour vérifier l'événement et la disponibilité des places.

Règle métier :
une inscription ne peut être confirmée que si des places sont disponibles.

MS3 — VENUE MANAGEMENT

Mission : gérer les salles et espaces universitaires.

Entité principale : Venue

id : Long
name : String
location : String
capacity : Integer
type : String
status : String

Endpoints principaux

POST /api/venues
GET /api/venues
GET /api/venues/{id}
PUT /api/venues/{id}
DELETE /api/venues/{id}
GET /api/venues/available

Interaction :
le service Events consulte Venues pour vérifier la disponibilité et la capacité d'une salle.

❇ MS4 — REVIEW MANAGEMENT

Mission : collecter les avis et évaluations des événements.

Entité principale : Review

id : Long
studentId : String
eventId : Long
rating : Integer
comment : String
createdAt : LocalDateTime

Endpoints principaux

POST /api/reviews
GET /api/reviews/event/{id}
GET /api/reviews/{id}
PUT /api/reviews/{id}
DELETE /api/reviews/{id}
GET /api/reviews/event/{id}/average

Interaction :
Reviews → Registrations pour vérifier qu'un étudiant est inscrit à l'événement.

❇ MS5 — ANALYTICS & STATISTICS

Mission : centraliser les indicateurs nécessaires au suivi des événements.

Fonctionnalités envisagées

Nombre total d'événements
Nombre d'inscriptions
Taux de participation
Notes moyennes
Statistiques par événement

Base de données : MongoDB

Communication : récupération des données depuis les services Events, Registrations et Reviews.

Architecture : agrégation des données sans accès direct aux bases des autres services. Les endpoints et le schéma des agrégats restent à définir.

NOS CHOIX TECHNOLOGIQUES

Technologie	Utilisation
Spring Boot	Développement des microservices REST
Spring Cloud Gateway	Routage des requêtes
Eureka	Découverte des services
Config Server	Configuration centralisée
OpenFeign	Communication REST interservices
Keycloak	Authentification et rôles
H2 / MySQL	Stockage relationnel
Optimization	Stockage des agrégats Analytics

Pourquoi cette architecture ?

Modularité, séparation des responsabilités, maintenabilité et évolution indépendante des services.

PRODUCT BACKLOG — UNIVERSITY EVENTS

User Story	Priorité	Service
Consulter les événements	Haute	Events
Créer un événement	Haute	Events
S'inscrire à un événement	Haute	Registrations
Gérer les salles	Haute	Venues
Déposer un avis	Moyenne	Reviews
Consulter les statistiques	Moyenne	Analytics

DIAGRAMME DE CAS D'UTILISATION

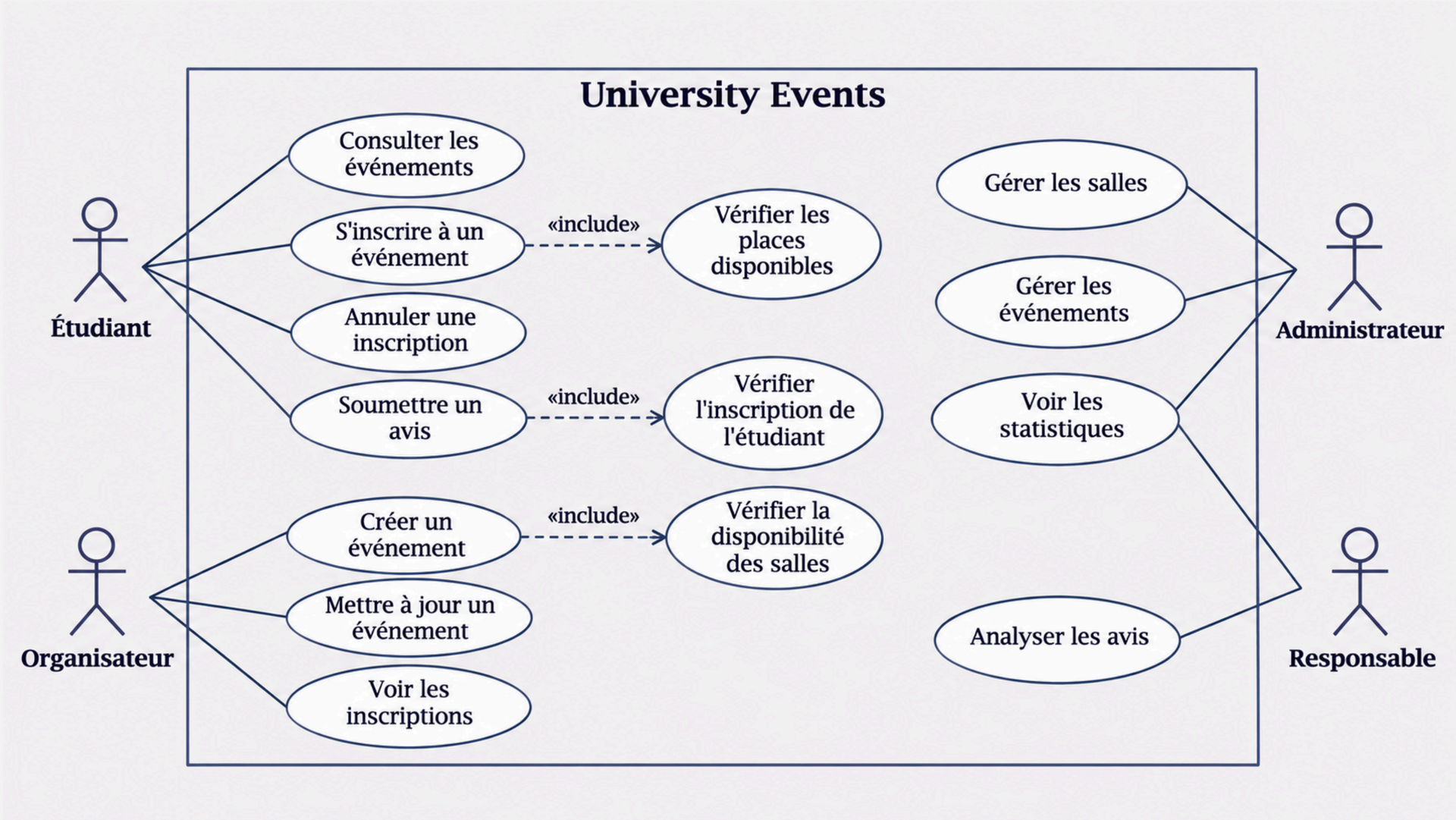
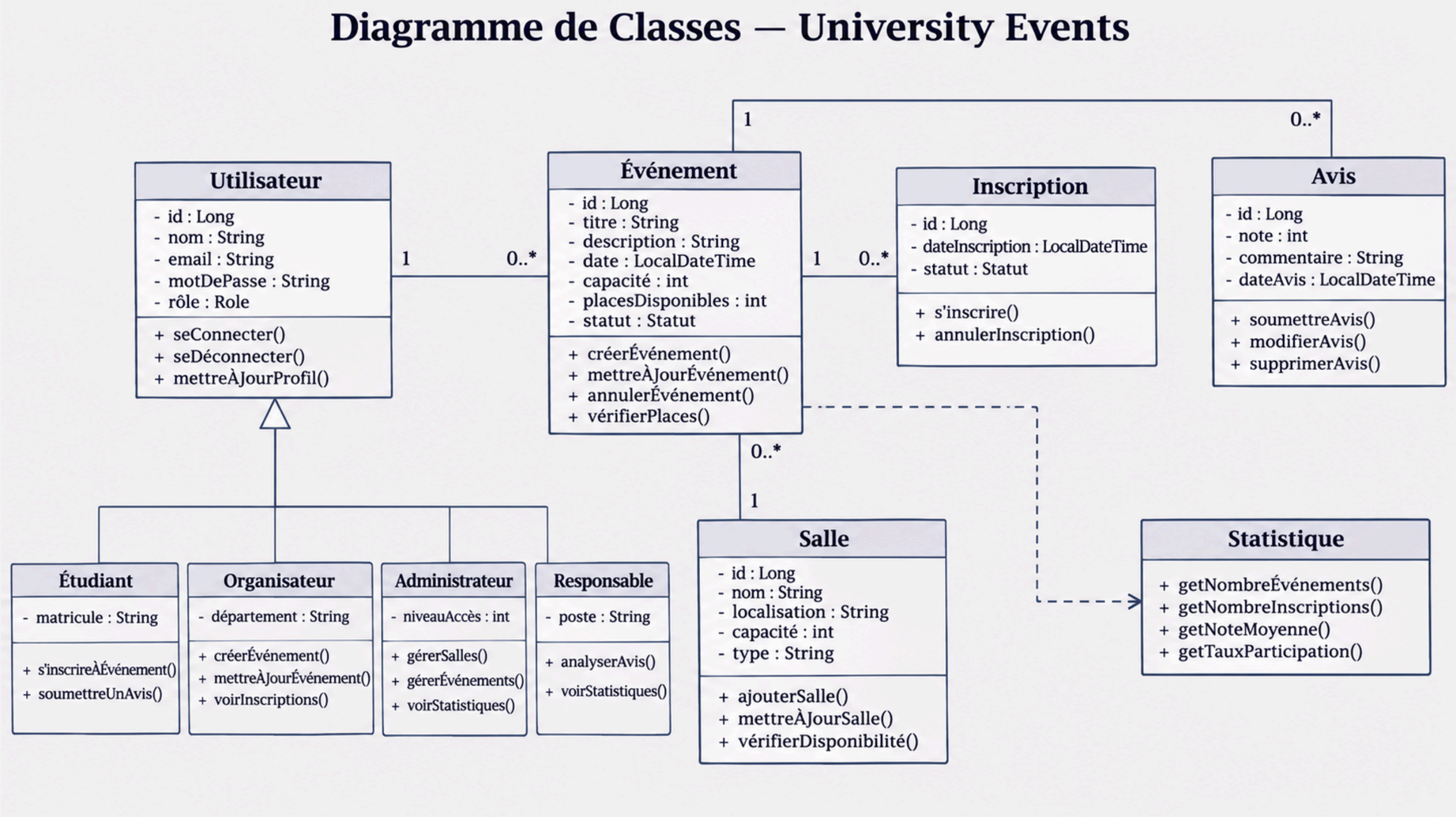


DIAGRAMME DE CLASSES



CONCLUSION

University Events propose une plateforme centralisée et sécurisée pour simplifier la gestion des événements universitaires, grâce à une architecture microservices moderne et évolutive.

MERCI

Pour Votre Attention